

Comparative Study of Deep Learning Approaches for Neural Style Transfer: Fast Feed-Forward Networks vs. Recent Arbitrary Style Transfer Methods

Navaneet Jayakrishnan
Roll No. B230448EC

Dept. of Electronics and Communication Engineering
National Institute of Technology Calicut
navaneet_b230448ec@nitc.ac.in

Abstract—Neural style transfer (NST) renders a content photograph in the artistic style of a reference image. Early optimisation-based methods produce high-quality results but are slow, which motivated feed-forward networks trained with perceptual losses and, more recently, arbitrary-style methods that need no per-style training. In this work I train a Fast Neural Style Transfer network (Fast-NST) from scratch on an MS-COCO subset with a WikiArt Expressionism style image, and compare it against two pretrained existing methods, AdaIN (ICCV 2017) and the transformer-based StyTr² (CVPR 2022). Using 30 held-out test images, I evaluate content loss, style (Gram) loss, SSIM, LPIPS and inference time. Fast-NST obtained the best content preservation (SSIM 0.54) and the fastest inference (3.7 ms/image), StyTr² produced the cleanest stylisations but was about 23× slower, and AdaIN gave the strongest texture transfer at the cost of structural distortion. An ablation on the style-loss weight confirms the expected content–style trade-off.

Index Terms—neural style transfer, perceptual loss, AdaIN, StyTr², image stylisation, deep learning

I. INTRODUCTION

Style transfer aims to combine the semantic content of one image with the visual style (colour palette, brush strokes, texture) of another. Gatys *et al.* showed that the feature statistics of a pretrained convolutional network can separate and recombine content and style [2]. Their method optimises the pixels of the output image for every content–style pair, which takes seconds to minutes per image.

Johnson *et al.* removed this cost by training a feed-forward image transformation network with the same perceptual losses, so that stylisation needs a single forward pass [1]. The price is that one network must be trained per style. Arbitrary style transfer methods remove this limitation. AdaIN aligns channel-wise feature statistics of the content to those of the style in a single layer [3], and StyTr² uses transformer encoders to model long-range content–style relationships [4].

Which of these families is preferable depends on the trade-off between stylisation quality, structure preservation and

speed. In this project I make this trade-off measurable. My contributions are:

- a from-scratch implementation and training of Fast-NST on MS-COCO with a WikiArt style;
- a quantitative comparison against two existing methods (AdaIN, StyTr²) using five measures: content loss, style loss, SSIM, LPIPS and inference time;
- a qualitative comparison and an ablation study of the style-loss weight.

II. PROBLEM DEFINITION

Let $\mathbf{x}_c \in [0, 1]^{3 \times H \times W}$ be a content image and $\mathbf{x}_s$ a style image. The task is to generate a stylised image $\mathbf{y}$ such that (i) $\mathbf{y}$ preserves the semantic structure of $\mathbf{x}_c$ and (ii) $\mathbf{y}$ reproduces the colour and texture statistics of $\mathbf{x}_s$.

For the trained model, a network f_W is learned so that $\mathbf{y} = f_W(\mathbf{x}_c)$ for a fixed style $\mathbf{x}_s$, by minimising

$$W^* = \arg \min_W \mathbb{E}_{\mathbf{x}_c} [\lambda_c \mathcal{L}_c + \lambda_s \mathcal{L}_s + \lambda_{tv} \mathcal{L}_{tv}], \quad (1)$$

where the terms are defined in Section III-C. For the arbitrary-style methods, a single model g produces $\mathbf{y} = g(\mathbf{x}_c, \mathbf{x}_s)$ for any style image without further training.

The objectives are: (1) select a dataset for content and style images and pre-process it; (2) train one deep model with a proper train/test split; (3) compare it with at least two recent existing methods using at least three quantitative measures; and (4) study the effect of the loss weighting through an ablation. There is no ground-truth stylised image, so evaluation relies on feature-based and perceptual measures computed against the content and style inputs.

III. METHODOLOGY

A. Datasets and Pre-processing

Content images. I use the MS-COCO 2017 validation split (5,000 images, randomly sampled with seed 42) and split it 90%/10% into 4,500 training images and 500 held-out test

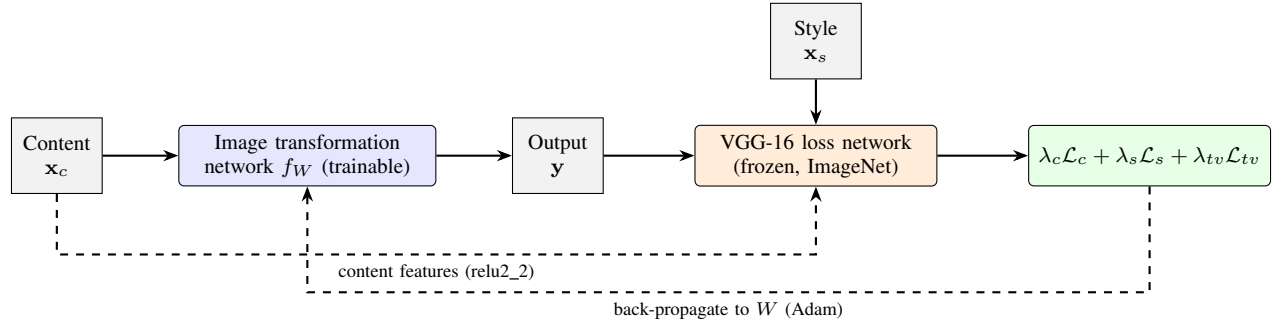


Fig. 1. Training pipeline of the Fast Neural Style Transfer model. Only the transformation network is updated; the VGG-16 network provides the content and style feature targets.



Fig. 2. Training style image (left) and samples of COCO content images.

images. The training images are used only to train Fast-NST; all reported metrics are computed on the first 30 images of the test set.

Style images. Style images come from the public WikiArt collection (huggan/wikiart), Expressionism category. A single style image was used both to train Fast-NST and as the style reference for all three methods at evaluation.

Pre-processing. Every image is converted to RGB, resized so that the shorter side is 256 pixels, centre-cropped to 256×256 and scaled to $[0, 1]$. Inside the VGG loss network the images are normalised with ImageNet mean and standard deviation.

Fig. 2 shows the style image and content samples used.

B. Fast Neural Style Transfer (Trained Model)

I implement the image transformation network of Johnson *et al.* [1]. It has three convolutional layers (a 9×9 convolution with 32 channels followed by two stride-2 3×3 convolutions with 64 and 128 channels), five residual blocks with 128 channels, two nearest-neighbour up-sampling convolution layers (64 and 32 channels) and a final 9×9 convolution to 3 channels. Reflection padding and instance normalisation are used throughout, ReLU activations follow every layer except the last, and a sigmoid keeps the output in $[0, 1]$.

C. Perceptual Loss

Let ϕ_j denote the activations of layer j of an ImageNet-pretrained VGG-16. The content loss is the mean squared error of the `relu2_2` features,

$$\mathcal{L}_c = \|\phi_{\text{relu2}_2}(\mathbf{y}) - \phi_{\text{relu2}_2}(\mathbf{x}_c)\|_2^2. \quad (2)$$

The style loss compares Gram matrices at `relu1_2`, `relu2_2`, `relu3_3` and `relu4_3`,

$$\mathcal{L}_s = \sum_j \|G_j(\mathbf{y}) - G_j(\mathbf{x}_s)\|_2^2, \quad G_j(\mathbf{x}) = \frac{\Phi_j \Phi_j^\top}{C_j H_j W_j}, \quad (3)$$

where $\Phi_j \in \mathbb{R}^{C_j \times H_j W_j}$ is the reshaped feature map. A total-variation term $\mathcal{L}_{tv}$ (mean absolute difference between neighbouring pixels) encourages spatial smoothness. I use $\lambda_c = 10^5$, $\lambda_s = 10^{10}$ and $\lambda_{tv} = 10^{-6}$.

D. Training Details

The network is trained with Adam (learning rate 10^{-3}), batch size 4, for 2 epochs over the 4,500 training images (2,250 iterations). Training took about 197 s on a single NVIDIA L4 GPU using PyTorch. The random seed was fixed to 42.

E. Existing Methods for Comparison

AdaIN [3] encodes both images with a VGG-19 encoder up to `relu4_1` and replaces the channel-wise mean and standard deviation of the content features with those of the style features,

$$\text{AdaIN}(\mathbf{f}_c, \mathbf{f}_s) = \sigma(\mathbf{f}_s) \frac{\mathbf{f}_c - \mu(\mathbf{f}_c)}{\sigma(\mathbf{f}_c)} + \mu(\mathbf{f}_s), \quad (4)$$

before a learned decoder maps the result back to image space.

StyTr² [4] tokenises content and style images into patch sequences, processes them with separate transformer encoders and a transformer decoder, and decodes the result with a convolutional decoder, so that content and style are related through attention rather than global statistics.

Both are used as *inference-only* baselines with the official pretrained weights released by their authors. Inputs are 256×256 for all methods, which is lower than the 512×512 resolution used by the StyTr² authors.

F. Evaluation Metrics

Given output $\mathbf{y}$, content $\mathbf{x}_c$ and style $\mathbf{x}_s$, I report:

- **Content loss:** $\mathcal{L}_c$ above (lower is better).
- **Style loss:** $\mathcal{L}_s$ above (lower is better).
- **SSIM** between $\mathbf{y}$ and $\mathbf{x}_c$ on RGB images (higher is better).

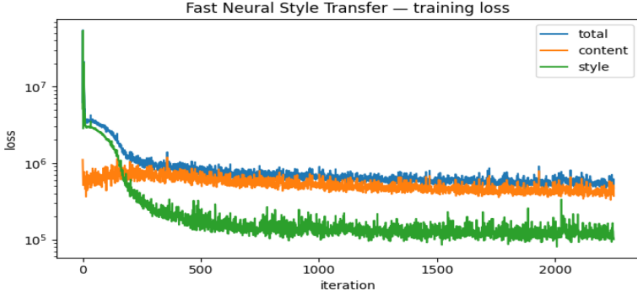


Fig. 3. Training loss of Fast-NST (log scale): total, content and style terms.

- **LPIPS** (AlexNet) between $\mathbf{y}$ and $\mathbf{x}_c$ (lower is better).
- **Inference time**: mean time per image over 5 repeated runs with GPU synchronisation.

G. Ablation Setup

To study the content–style trade-off I retrain Fast-NST with $\lambda_s \in \{10^8, 10^9, 10^{10}, 10^{11}\}$ and λ_c fixed, using 1 epoch on a 2,000-image subset (500 iterations) per setting, and evaluate the mean content and style loss on 10 test images.

IV. RESULTS AND DISCUSSION

A. Training Behaviour

Fig. 3 shows the training curves. The style term falls from above 10^7 to about 10^5 within the first few hundred iterations, after which the content term decreases slowly. The total loss went from about 6.2×10^6 at the first iteration to roughly $5\text{--}7 \times 10^5$ at the end. The curves are noisy from batch to batch because each batch contains different content images, but the trend is steadily downward and stable.

B. Quantitative Comparison

Table I and Fig. 4 summarise the metrics over 30 test images (mean $\pm$ standard deviation).

Content preservation. Fast-NST scores best on all three content-fidelity measures (content loss, SSIM, LPIPS), followed by StyTr². AdaIN is clearly worst (SSIM 0.18 against 0.54), which is consistent with its design: matching only channel-wise mean and variance offers no mechanism to protect spatial structure.

Style strength. Fast-NST has the lowest style loss (1.3×10^{-5}), followed by AdaIN (1.6×10^{-5}) and StyTr² (2.1×10^{-5}). The gap between Fast-NST and AdaIN is small relative to the spread across images (standard deviation of Fast-NST $\approx 3.6 \times 10^{-6}$), so it should not be over-interpreted. The higher style loss of StyTr² agrees with its visibly subtler stylisation. Since SSIM, LPIPS and content loss are measured against the *content* image, a method that stylises less will score better on them; these numbers therefore have to be read together with the style loss and the images.

Efficiency. Fast-NST needs 3.7 ms per image, AdaIN 6.8 ms ($1.8\times$ slower) and StyTr² 84.5 ms ($\approx 23\times$ slower than Fast-NST and $\approx 12\times$ slower than AdaIN), because StyTr² runs a VGG encoder plus transformer encoders and decoder at test time.

C. Qualitative Comparison

Fig. 5 shows three test images. Fast-NST preserves object shapes and thin structures (bat, players, chalk lines) but shows faint repetitive grid-like texture in flat regions such as the ground and blurred background, a known artefact of feed-forward networks with up-sampling layers. AdaIN transfers the strongest texture and colour, but boundaries are distorted and the players show ghosting. StyTr² gives the cleanest, most content-faithful outputs, with weaker colour and texture transfer. Grey-scale inputs (second row) are recoloured by all three methods towards the golden palette of the style image.

D. Ablation: Style-Loss Weight

Table II and Fig. 6 show the effect of λ_s . Increasing the style weight monotonically trades content fidelity for style fidelity: over three orders of magnitude the style loss falls about $26\times$ while the content loss rises about $5\times$. Small weights yield outputs close to the content image with weak stylisation, while very large weights let style dominate and degrade content structure. I used $\lambda_s = 10^{10}$ in the main experiment as a balanced setting. The ablation run at 10^{10} has a higher content loss (5.28) than the main model (4.28) because it was trained for only one epoch on a subset, which indicates that longer training further improves content preservation.

E. Limitations

- Fast-NST is trained on the very style used for evaluation, whereas AdaIN and StyTr² have never seen it. Its advantage in style loss is therefore partly expected, and it must train a new network per style, unlike the other two methods.
- Only one style image, 30 test images and short training (2 epochs on 4,500 images) were used, so the results are indicative rather than conclusive.
- StyTr² was evaluated at 256×256 although it was trained at 512×512 , which may slightly disadvantage it.
- The content and style losses use the same VGG features on which Fast-NST was trained, which may favour it. SSIM and LPIPS, being independent of that network, partly mitigate this.
- Absolute style-loss values are small because Gram matrices are normalised by $C_j H_j W_j$; only relative comparisons are meaningful.

V. CONCLUSION

A Fast Neural Style Transfer network was trained from scratch and compared with two pretrained arbitrary-style methods, AdaIN and StyTr², using content loss, style loss, SSIM, LPIPS and inference time. Fast-NST achieved the best content preservation, competitive stylisation and the fastest inference, at the cost of being tied to one style. StyTr² preserved structure better than AdaIN and gave the cleanest images but was by far the slowest, while AdaIN offered a fast and flexible middle ground with the weakest structural fidelity. The ablation confirmed the expected trade-off controlled by the style-to-content weight ratio.

TABLE I
QUANTITATIVE COMPARISON ON 30 HELD-OUT COCO TEST IMAGES (EXPRESSIONISM STYLE, 256×256 , NVIDIA L4). BEST VALUE IN BOLD.

Method	Content loss ↓	Style loss ($\times 10^{-5}$) ↓	SSIM ↑	LPIPS ↓	Time (ms) ↓
Fast-NST (mine, trained)	4.28 ± 0.77	1.3 ± 0.4	0.537 ± 0.092	0.454 ± 0.099	3.7 ± 0.2
AdaIN [3] (2017)	8.68 ± 1.66	1.6 ± 0.1	0.182 ± 0.032	0.574 ± 0.079	6.8 ± 0.1
StyTr ² [4] (2022)	5.82 ± 1.23	2.1 ± 0.3	0.458 ± 0.078	0.518 ± 0.092	84.5 ± 0.5

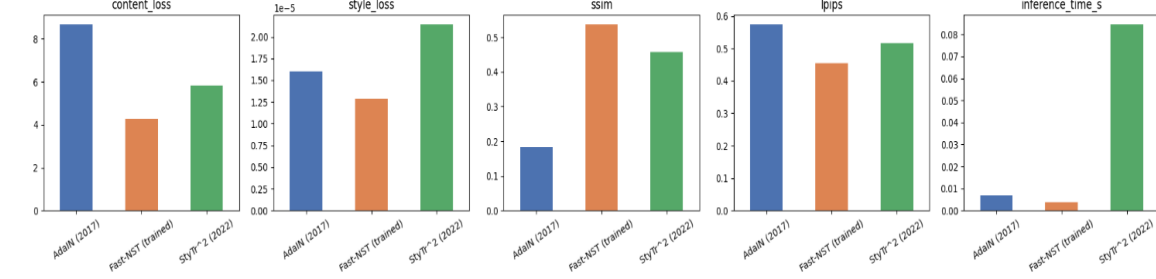


Fig. 4. Mean value of each metric for the three methods (content loss, style loss, SSIM, LPIPS and inference time).



Fig. 5. Qualitative comparison. Columns: content, style, Fast-NST (mine), AdaIN, StyTr².

TABLE II
EFFECT OF THE STYLE WEIGHT λ_s (1 EPOCH, 2,000 IMAGES, 10 TEST IMAGES).

λ_s	Content loss ↓	Style loss ↓
10^8	1.61	1.6×10^{-4}
10^9	2.25	9.6×10^{-5}
10^{10}	5.28	1.4×10^{-5}
10^{11}	8.50	6.0×10^{-6}



Fig. 6. Content and style loss as a function of the style weight λ_s .

REFERENCES

- [1] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *Proc. Eur. Conf. Comput. Vis.* (ECCV), 2016, pp. 694–711.

- [2] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2414–2423.
- [3] X. Huang and S. Belongie, "Arbitrary style transfer in real-time with adaptive instance normalization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 1501–1510.
- [4] Y. Deng, F. Tang, W. Dong, C. Ma, X. Pan, L. Wang, and C. Xu, "StyTr²: Image style transfer with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 11326–11336.